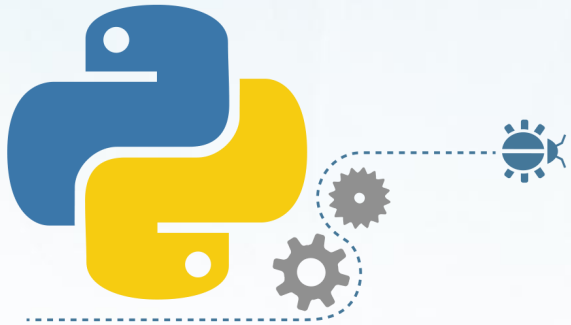
TRƯỜNG ĐẠI HỌC
SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
HCMC University of Technology and Education

KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN HỆ THỐNG THÔNG TIN

NHẬP MÔN LẬP TRÌNH PYTHON (IPPA233277)

PHÂN TÍCH DỮ LIỆU
THƯ VIỆN NUMPY



GV. Trần Quang Khải

1. Nắm và hiểu được phân tích dữ liệu là gì?
2. Nắm và vận dụng được các hàm hỗ trợ thư viện Numpy

1. Giới thiệu về phân tích dữ liệu
2. Tiến trình phân tích dữ liệu
3. Phân loại phân tích dữ liệu
4. Các gói phân tích dữ liệu
5. Thư viện numpy

- Trong chương trước chúng ta đã tìm hiểu về list – một cấu trúc dữ liệu cho phép lưu trữ mảng. List đơn giản và có đầy đủ chức năng cơ bản của mảng. Tuy nhiên, khi chứa mảng số với nhiều phần tử, list có tốc độ thực thi và hỗ trợ tính toán không bằng numpy arrays – một cấu trúc dữ liệu dạng mảng của thư viện numpy.
- NumPy (phát âm là /ˈnʌmpaɪ/) viết tắt cụm từ **numerical python** là một thư viện cho Python, hỗ trợ làm việc với các mảng lớn, nhiều chiều, và cung cấp các hàm toán học cấp cao hoạt động trên các mảng này
- Thường được sử dụng tính toán đại số trên ma trận & vector

- Cài đặt numpy : **pip install numpy**
- Khai báo thư viện : **import numpy as np**

Cần cài đặt thư viện trước khi khai báo, nếu không khi thực thi lệnh import sẽ báo lỗi không tìm thấy thư viện.

Lưu ý:

✓ Trường hợp chưa có thư viện pip cần cài đặt pip trước

✓ Sử dụng import sys; print(sys.path) → xem đường dẫn đến thư mục Scripts của Python

✓ Import đường dẫn này vào biến môi trường (Environment Variables)

✓ Khởi động lại máy tính

- **Tạo mảng**

arr = np.array([value1, value2,…], dtype = <datatype_name>)

Ví dụ:

Mảng 1 chiều: arr = np.array([1, 2, 3]) # [1, 2, 3]

Mảng 2 chiều: arr = np.array([(1, 2), (3, 4)]) # [[1, 2]

[3, 4]]

Mảng 3 chiều: arr = np.array(([[), ()], [(), ()], [(), ()]))

Lưu ý:

- **axis: 0|cột – 1|hàng**
- **shape: hàng x cột x …**



- **Hàm dựng sẵn**
  - ✓ np.zeros(shape) # ma trận toàn phần tử **0**
  - ✓ np.ones(shape) # ma trận toàn phần tử **1**
  - ✓ np.arange(start, stop, step) # ma trận $start \leq value < stop$
  - ✓ np.full(shape, fill_value) # ma trận toàn phần tử **fill_value**
  - ✓ np.random.seed(id_memory) # giữ nguyên kết quả random
  - ✓ np.random.randn(shape) # ma trận ngẫu nhiên
- **Ma trận vuông**
  - ✓ np.eye/identity(n) # ma trận đơn vị n
  - ✓ np.diag(arr) # ma trận đường chéo hoặc trả về đường chéo



- **Đọc và lưu numpy từ file**
  - ✓ np.save/load(file, arr) # lưu/đọc ma trận vào file .npy [.npz]
  - ✓ np.savez(file, arr1, arr2,…) # lưu nhiều ma trận vào file .npz
  - ✓ np.savetxt/loadtxt(file, arr) # lưu/đọc ma trận vào file text
- **Truy xuất ma trận | array[i][j] ~ array[i, j]**

Cú pháp:

indice_start : (indice_end + 1)

-num_indices: **#** lấy số lượng num_indices ở vị trí cuối cùng

:num_indices **#** lấy một số lượng num_indices ở vị trí đầu tiên

Cho đoạn code sau:

```
import numpy as np
X = np.array([[1, 2, 3, 4],
              [4, 5, 6, 2],
              [7, 8, 9, 1],
              [2, 5, 1, 5]])
print(X[1, 2])       # Truy cập vào phần tử thuộc dòng 1 và cột 2
print(X[1:3, 1:3])   # Truy cập vào các dòng từ 1 đến 2 và các cột từ 1 đến 2
print(X[:2, :2])     # Truy cập vào 2 dòng, 2 cột đầu tiên
print(X[-2:, -2:])   # Truy cập vào 2 dòng, 2 cột cuối cùng
print(X[[0, 2], :]) # Truy cập vào dòng 0 và 2. Cách lấy indices không liên tục
```

Cho đoạn code sau:

```
import numpy as np
X = np.array([[1, 2, 3, 4],
              [4, 5, 6, 2],
              [7, 8, 9, 1],
              [2, 5, 1, 5]])
```

Hãy dùng cách thức truy cập để lấy ra các giá trị sau:

a. **[[2, 3] [5, 6]]**

b. **[[1, 3] [4, 6] [7, 9]]**

c. **[1, 5, 9, 5]**

d. **[[1, 4] [4, 2]]**

e. **[[2, 3] [5, 6] [8, 9] [5, 1]]**



- **Chuyển vị các chiều**
  - ✓ np.transpose(array, (d_i, d_j, …, d_n))
- **Thay đổi shape của ma trận**
  - ✓ np.reshape(new_shape) hoặc arr.reshape(new_shape) với giá trị **-1** numpy sẽ tự tính chiều còn lại
- **Ghép ma trận**
  - ✓ np.concatenate((arr1, arr2, …, arrn), axis=position)
  - ✓ np.stack/vstack/hstack/dstack([arr1, arr2, …, arrn])
- **Tách ma trận**
  - ✓ arr1, arr2 = np.split/vsplit/hsplit/dsplitk(arr)
- **Mở rộng ma trận**
  - ✓ np.expand_dims(arr, axis=position)



Cho mảng B = np.array(np.arange(12))

Xem số phần tử của mỗi chiều ma trận

print("B.shape: ", B.shape)

Biến đổi số chiều của ma trận

B.reshape(2, 6) hoặc B.reshape(2, -1) hoặc np.reshape(B, (2, 6)) hoặc np.reshape(B, (2, -1))

a. **Phát sinh mảng A gồm 2352 phần tử ngẫu nhiên số nguyên không âm trong đoạn [1, 20]**
b. **Phát sinh mảng B gồm 2352 phần tử ngẫu nhiên số nguyên không dương trong đoạn [-20, -1]**
c. **Xem thông tin các chiều của mảng trên**
d. **Tạo ma trận C, D lần lượt từ mảng A, B về ma trận 3 chiều với (28, 28, 3)**
e. **Nối ma trận C, D theo dòng**
f. **Đặt 2 ma trận C, D lên nhau**
g. **Ghép 2 ma trận C, D theo chiều ngang, dọc, rộng**



- **Phép toán ma trận**
  - ✓ arr.T # phép chuyển vị m x n → n x m
  - ✓ np.linalg.pinv(arr) # ma trận nghịch đảo ma trận vuông
  - ✓ np.linalg.det(arr) # định thức của ma trận
  - ✓ np.trace(arr) # tổng giá trị trên đường chéo chính
  - ✓ np.linalg.norm(arr, ord='fro') # chuẩn Frobenious
  - ✓ np.dot(arr1, arr2) | arr1.dot(arr2) | A@B # nhân hai ma trận
  - ✓ arr * value # nhân ma trận với giá trị

- **Hàm trên numpy**
  - ✓ min/max/mean/sum([axis = position])
  - ✓ minimum/maximum(arr1, arr2, …, arrn)
  - ✓ argmax/argmin(arr, axis = position) # vị trí có giá trị max/min
  - ✓ argsort(+/-arr, axis = position) # vị trí có giá trị tăng/giảm dần
  - ✓ np.exp(arr) # tính số mũ cơ số tự nhiên e, $f(x) = e^x$
- **Giữ nguyên shape**
  - ✓ np.*functionX*(…, keepdims=True) # chiều tiêu giảm = 1

Cho 2 ma trận A = np.array([[1, 3, 6], [2, 7, 9]]) và B = np.array([[0, 4, 5], [1, 6, 10]])

a. Tìm phần tử nhỏ nhất trong A

b. Tìm phần tử nhỏ nhất của A theo từng dòng

c. Tìm phần tử nhỏ nhất của A theo từng cột

d. Tìm các phần tử maximum/minimum của A và B ở cùng vị trí indice

e. Tìm ra indice có giá trị lớn nhất của các dòng trong ma trận A

f. Tìm ra indice của các cột theo thứ tự tăng/giảm dần của ma trận A

Bài tập: Cho ma trận bên dưới là đầu ra có kích thước NxCtrong đó N là số quan sát và C là số classes. Mỗi dòng của ma trận là một phân phối xác suất của một quan sát. Thống kê kết quả nhãn thuộc về mỗi loại.

```
B = np.array([[0.1, 0.2, 0.7],
              [0.6, 0.4, 0.0],
              [0.1, 0.5, 0.4],
              [0.3, 0.3, 0.4],
              [0.1, 0.8, 0.1]])
```




ĐIỂM DANH

m2



TRẮC NGHIỆM

d12



- ✓ Họ tên : Trần Quang Khải
- ✓ Email : khaitq@hcmute.edu.vn
- ✓ Zalo (mã Qr)

